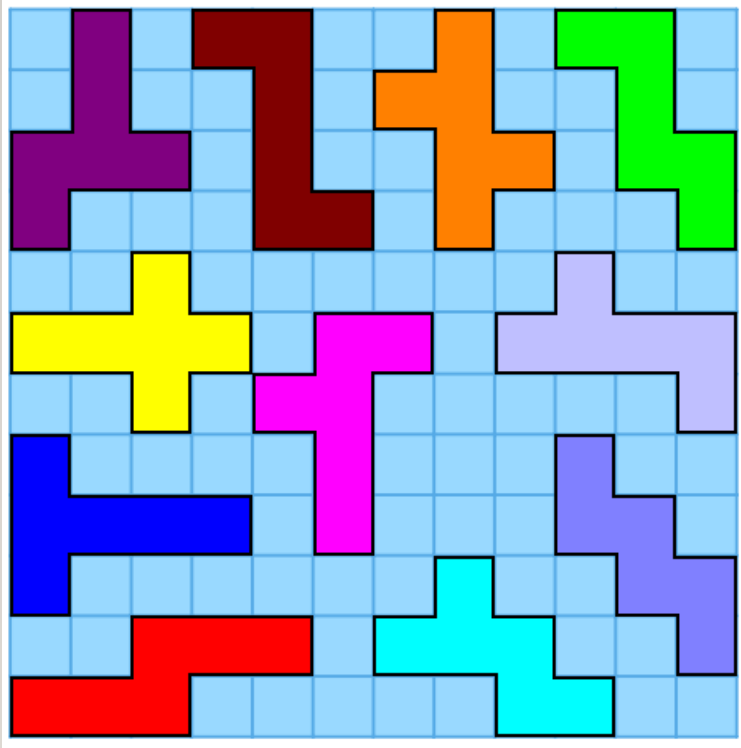


UNTOUCHABLE 11

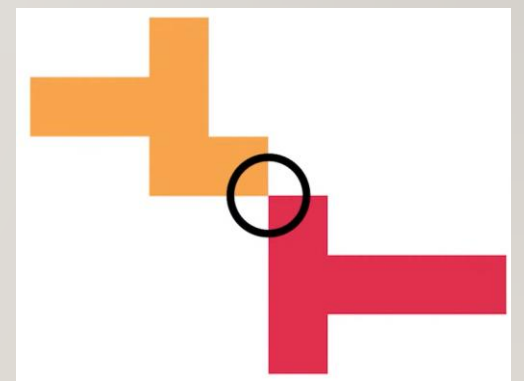
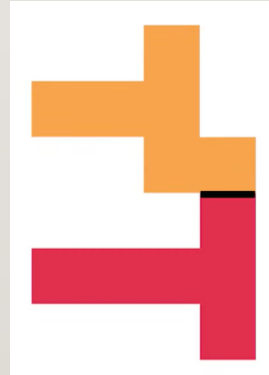
吳展宇

規則

像這就不是一個符合規則的放置，因為黃色和粉紅色的拼圖塊的角接觸到了



有11塊不同的拼圖塊，每一塊剛好都是一個正方體的平面展開圖，棋盤大小共有3種，依照難度依序為 9×17 、 10×15 、 12×12 ，目標要將所有拼圖塊都放進棋盤裡，每一個拼圖塊都可以旋轉、翻面，但任兩塊拼圖塊都不能有任何接觸，不管是邊還是角。



以9*17為例

CNF式子建立(模型A)

- 符號：P=拼圖塊的總數量=11、M=棋盤的總格子數=153、D=一個格子的鄰居數=8、N=拼圖塊的總放置可能數=6008、k=平均每塊拼圖塊的放置數=N/P=6008/11=546
- 變數：v(p,o,x,y) 共6008個
- 每塊拼圖至少一次： $(v_1+v_2+\dots+v_k)(v_{k+1}+v_{k+2}+\dots+v_{2k})\dots(v_{N-k+1}+v_{N-k+2}+\dots+v_N)$
 $O(P)$
- 每塊拼圖至多一次： $(-v_1+-v_2)(-v_1+-v_3)\dots(-v_1+-v_k)(-v_2+-v_3)\dots(-v_{k-1}+-v_k)$
 $O(P*k^2)=O(N^2/P)$
- 不可相鄰or重疊： $(-v_1+-v_{k+1})\dots(-v_1+-v_N)(-v_2+-v_{k+1})\dots(-v_k+-v_N)(-v_{k+1}+-v_{2k+1})\dots(-v_{N-k}+-v_N)$
 $O(P^2*k^2)=O(N^2)$
- 總時間複雜度： $O(P)+O(N^2/P)+O(N^2)=O(N^2)$ 約 10^7

改善了模型A中的 $O(N^2)$ 的問題

以 $9*17$ 為例

改進方法(模型B)

- 引進新的變數： $c(x,y,p)$ 共 $153*11=1683$ 個
- 每塊拼圖至少一次和至多一次跟模型A一樣 $O(P) + O(N^2/P)$
- 不可重疊： $(-c_{1,1,1} + -c_{1,1,2})(-c_{1,1,1} + -c_{1,1,3}) \dots (-c_{1,1,1} + -c_{1,1,11}) \dots (-c_{9,17,10} + -c_{9,17,11})$
 $O(M*P^2)$
- 不可相鄰： $(-c_{1,1,1} + -c_{1,2,2})(-c_{1,1,1} + -c_{2,1,2}) \dots (-c_{1,1,1} + -c_{2,2,2}) \dots$
 $O(M*(D/2)*P^2)$
- $V \Rightarrow C$ ： $(-v_1 + c_{1,1,1}) \dots (-v_N + c_{9,17,11})$
 $O(N*6) = O(N)$
- $C \Rightarrow V$ ： $(-c_{1,1,1} + v_1 + v_4 + v_{13} + \dots) \dots (-c_{9,17,11} + v_{567} + v_{621} + v_{3215} + \dots)$
 $O(M*P)$
- 總時間複雜度： $O(P) + O(N^2/P) + O(M*(D/2)*P^2) + O(N) + O(M*P) = O(N^2/P)$ 約 10^6

改進方法(模型C)

- 每塊拼圖至少一次、不可重疊、不可相鄰、 $v \Rightarrow c$ 、 $c \Rightarrow v$ 和模型B一樣

$$O(P) + O(M \cdot P^2) + O(M \cdot (D/2) \cdot P^2) + O(N) + O(M \cdot P)$$

- 每塊拼圖至多一次：使用Sequential Counter Encoding

加入 $(k-1)*11$ 個變數 s

$$(-v_1 + s_1)(-v_2 + s_2) \dots (-v_{k-1} + s_{k-1})$$

$$(-s_1 + s_2)(-s_2 + s_3) \dots (-s_{k-2} + s_{k-1})$$

$$(-v_2 + s_1)(-v_3 + s_2) \dots (-v_k + s_{k-1})$$

$$O(P \cdot (3k-4)) = O(P \cdot k) = O(N)$$

- 總時間複雜度： $O(P) + O(M \cdot P^2) + O(M \cdot (D/2) \cdot P^2) + O(N) + O(M \cdot P) + O(N) = O(M \cdot (D/2) \cdot P^2)$ 約 10^5

這三種棋盤大小的拼圖都有不只一個的解，所以用不同的模型去解可能會得到不同的放置方法

如果想要解出所有的解，可以在每次解出一個解後，加入一個子句來把這個解給排除掉

結果



在9*17的例子中可以看到隨著採用改善後的模型後，解出一個解所花費的時間越來越短；在12*12的例子中也能看到在用了模型C後才解的出解，用其他兩個模型則花了超過24小時的時間都解不出任何解；但在15*15的例子中可以看到用模型C所要花費的時間反而比模型B還要長，我認為這可能是因為sat solver在用模型C解的時候的運氣比較不好，所以才會花比較久的時間

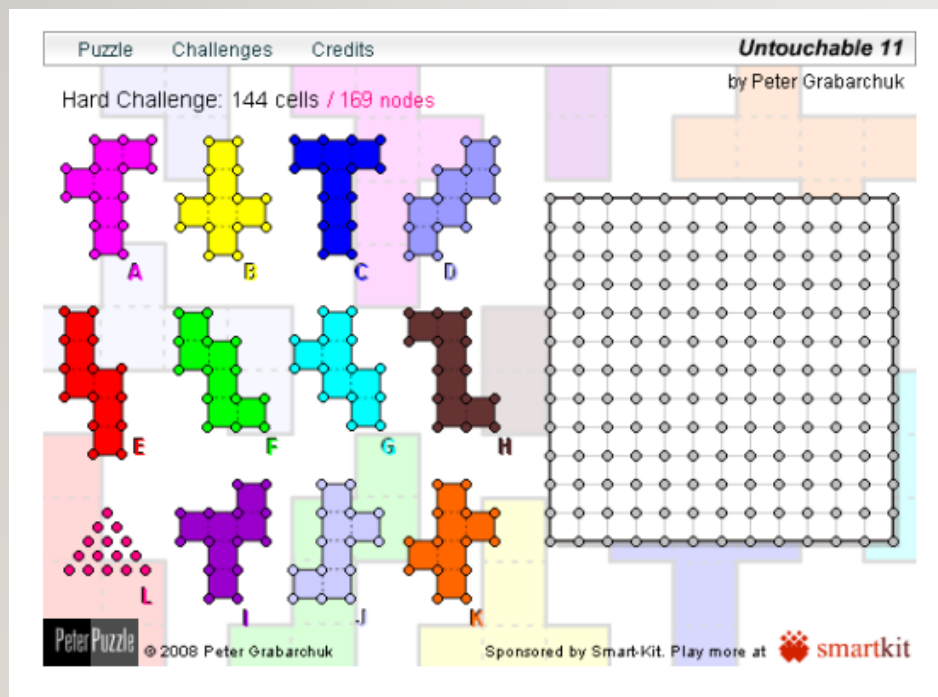
花費時間比較

使用CaDiCaL SAT Solver

因為經過實測後發現用mini sat解的速度會比較慢

	9*17	10*15	12*12
模型A	7min 24min(用minisat)	306min	解不出來
模型B	3min 14min(用minisat)	4min	解不出來
模型C	1min	25min	208min

未來展望



在這篇論文中有提到可以把這個問題映射為一個新的問題，在這個問題中只看拼圖塊周圍的那些頂點，棋盤也從原本的 12×12 的格子變成 13×13 的點，規則也從拼圖不能接觸變為任兩塊拼圖的頂點不要重合就好了，這篇論文的作者用原來的方法花了24天才找到 12×12 難度的所有7個解，而用了映射後的方法後只花了不到1個小時就找到了所有的7個解

<https://www.gathering4gardner.org/g4g13gift/puzzles/HoffCarl-GiftExchange-FromUntouchable11toHazmatCargo-G4G13.pdf>